GET TECHY WITH LUCKY

TECH PULSE INSIDER

WEB DEVELOPMENT MASTERCLASS

PROJECT GUIDE & BRIEF

The Capstone Project

Plan, Design, Build, Test, Deploy, Document and Defend a Complete Application

WEEK 08 - FULL-SCALE CAPSTONE PROJECT

BUILD. TEST. DOCUMENT. DEPLOY. PRESENT.

Prepared for: Get Techy With Lucky / Tech Pulse Insider

Instructor: Lucky Nakola

Learn • Build • Connect • Grow

Course Information

Field	Detail
Program	Web Development Masterclass
Provider	Get Techy With Lucky / Tech Pulse Insider
Instructor	Lucky Nakola
Week	Week 08 of 08
Module	Full-Scale Capstone Project
Estimated study time	10-12 hours
Builds on	Weeks 01-07 - the entire masterclass
Format	Read -> Practise -> Quiz -> Build -> Submit
Assessment	Weekly quiz (auto-scored) and a practical assignment submitted via GitHub

Learning Objectives

By the end of this week you should be able to:

1. Scope a project with a clear problem statement, target user and defensible boundary.
2. Translate a problem into functional and non-functional requirements.
3. Design a system architecture and a normalised database, and document both.
4. Build a complete full-stack application using everything from Weeks 1 to 7.
5. Test deliberately, including invalid input, edge cases and authorization bypass attempts.
6. Deploy the application securely and document how to run and maintain it.
7. Present the work and justify the decisions behind it under questioning.

Prerequisites

Before starting this week, make sure you can already do the following:

- Completed Weeks 1 to 7, including the weekly assignments.
- Comfortable with HTML, CSS, JavaScript, PHP, MySQL and Git.
- Able to deploy a project and configure it with environment variables.
- Able to write prepared statements and hash passwords correctly.

Table of Contents

This guide is written to be read in order. Each chapter builds on the one before it.

1. Introduction - What the Capstone Is For

2. Choosing a Project You Can Actually Finish
3. Stage 1 - Proposal and Problem Statement
4. Stage 2 - Requirements
5. Stage 3 - System and Interface Design
6. Stage 4 - Database Design
7. Stage 5 - Development
8. Stage 6 - Testing
9. Stage 7 - Deployment
10. Documentation and the README
11. Presenting and Defending Your Work
12. Important Terminology
13. Common Mistakes and How to Avoid Them
14. Security Checklist Before You Submit
15. Marking Rubric
16. Knowledge Check - Readiness Quiz
17. Project Brief - The Capstone Specification
18. Summary and What Comes Next

Chapter 1 - Introduction - What the Capstone Is For

For seven weeks you have built things where the requirements were given to you and the problem had a known answer. The capstone removes both. You choose the problem, decide what 'finished' means, and live with the consequences of your own design decisions. That is what professional work is.

It is also, practically, the most important thing you will produce in this course. When you apply for your first role, nobody will ask about your quiz scores. They will open your GitHub, read one README, and click one live link. This project is that link.

WHY THIS MATTERS

The capstone is graded on completeness, correctness and clarity - not ambition. A booking system that does four things properly, is secure, is deployed and is documented will score far higher than a social network where nothing quite works. Choose accordingly.

The Seven Stages

The student portal tracks your capstone through seven stages, and each has a section in this guide. Update your progress honestly as you go - it is how your instructor knows where to help before you get stuck rather than after.

Stage	What you produce	Roughly
1. Proposal	Problem statement, target user, scope	Day 1
2. Requirements	User stories with acceptance criteria	Day 1-2
3. Design	Wireframes, user flows, architecture	Day 2-3
4. Database	ER diagram and schema.sql	Day 3
5. Development	The working application	Day 4-9
6. Testing	TESTING.md with results and fixes	Day 9-10
7. Deployment	Live URL, documentation, presentation	Day 10-12

Chapter 2 - Choosing a Project You Can Actually Finish

The most common capstone failure is not weak code. It is choosing something too large, spending nine days building three quarters of it, and submitting something that does not work end to end. Pick a problem small enough that you can finish it, then make what you finish excellent.

Three Tests for a Good Capstone Idea

8. Can you describe the problem in two sentences to somebody who is not a developer?
9. Can you name a specific real person or organisation who has this problem?
10. Can you list the core user journey in five steps or fewer?

If any answer is no, the idea is not ready. Narrow it until all three are yes.

Project Ideas That Work Well

Project	Core entity	Why it works
Salon or clinic booking system	Booking	Clear roles, obvious CRUD, real scheduling logic
School fee tracker	Payment	Strong reporting angle, genuine local need
Small shop inventory manager	Product	Stock levels give real business rules
Church or SACCO member portal	Member	Roles and permissions matter naturally
Community events noticeboard	Event	Public plus authenticated areas, simple to scope
Farm produce marketplace	Listing	Two user types with different needs
Tutoring session manager	Session	Scheduling plus attendance plus reporting
Local delivery tracker	Delivery	Status workflow is a natural state machine

COMMON MISTAKE

Avoid anything requiring payment processing, live chat, video, machine learning or a mobile app. Each of those will consume your entire time budget and leave the core system unfinished. If your idea needs payments, model the payment as a record with a status and say so in the README.

Chapter 3 - Stage 1 - Proposal and Problem Statement

Write this before you write any code. It takes an hour and it is the difference between building something and building the right something.

```
PROBLEM STATEMENT

Salons in Kisumu take bookings over WhatsApp and record them in a paper
diary. Messages are missed when the phone is busy, double bookings happen
several times a week, and the owner cannot see the day's schedule without
the physical book.

TARGET USERS
  Primary:   Salon owners with 1-3 staff, comfortable with a smartphone
             but not with computers.
  Secondary: Customers, mostly on mid-range Android phones over mobile data.

PROPOSED SOLUTION
A web application where customers book an available slot themselves and
the owner sees the whole day's schedule on one screen.

IN SCOPE
  - Customer registration and login
  - Browsing services and available slots
```

```
  - Booking, viewing and cancelling a booking
  - Owner dashboard showing today's and this week's bookings
  - Owner management of services and staff

OUT OF SCOPE (and why)
  - Online payment        - out of time budget; cash on arrival is current practice
  - SMS reminders         - requires a paid gateway
  - Native mobile app     - the responsive web app covers the need

SUCCESS CRITERIA
A customer can book a slot in under a minute, and the owner can see the
day's schedule without the paper diary.
```

A complete proposal. Note that the out-of-scope section gives reasons - that is what makes it a decision rather than an omission.

BEST PRACTICE

The out-of-scope list is the most valuable part of this document. It is what you point at when you are tempted, on day six, to add 'just one more feature'.

Chapter 4 - Stage 2 - Requirements

Turn the proposal into a list of specific, checkable statements. User stories work well because they keep the user's purpose attached to the feature.

```
As a CUSTOMER I want to see available slots for a service
so that I can choose a time that suits me.

  Acceptance criteria:
    - Only future slots are shown
    - Slots already booked do not appear
    - Slots are grouped by day
    - The list works on a 360px screen

As an OWNER I want to see today's bookings on one screen
so that I can prepare for the day without the paper diary.

  Acceptance criteria:
    - Bookings are ordered by start time
    - Each shows customer name, service and time
    - Cancelled bookings are visually distinct
    - The page loads in under two seconds
```

Acceptance criteria are what turn a vague story into something you can test.

Non-Functional Requirements

These are easy to forget and they are where marks are quietly lost. Write down at least four.

- **Performance.** Pages load in under three seconds on a 3G connection.
- **Responsiveness.** Fully usable from 320px to 1440px.
- **Security.** Passwords hashed, all queries prepared, all output escaped.

- **Accessibility.** Contrast passes, every form field labelled, keyboard navigable.
- **Reliability.** Invalid input never produces a crash or a blank page.

Chapter 5 - Stage 3 - System and Interface Design

Architecture

```
BROWSER                    SERVER                     DATABASE
---------                  ------                     --------
HTML / CSS                 PHP
JavaScript      HTTPS      - routing        SQL       MySQL
(Tailwind)   <---------->  - validation  <-------->   - users
                           - auth check               - services
                           - business rules           - bookings
                           - queries (PDO)            - staff

Presentation tier          Application tier           Data tier
```

Three-tier architecture. Draw your own version - it forces you to decide where logic lives.

WHY THIS MATTERS

The single most important line in that diagram is that validation and authorization sit in the application tier. Anything enforced only in the browser is a suggestion, because the browser is under the user's control, not yours.

Wireframes and User Flows

Sketch every screen before building it, on paper if you like. A wireframe takes five minutes and saves an hour of rebuilding. Then write out the main user flow as a sequence, and check that every screen it needs exists in your sketches.

```
BOOKING FLOW

  Home  ->  Services  ->  Pick a slot  ->  [logged in?]
                                                |
                                      no  ->  Login/Register  ->  back to slot
                                                |
                                      yes ->  Confirm  ->  Success  ->  My Bookings
```

Writing the flow out reveals the branches you would otherwise discover mid-build.

Chapter 6 - Stage 4 - Database Design

Design the database before writing queries. Changing a schema after you have built ten pages against it is the most painful rework in this project.

11. List the nouns in your requirements. Those are your candidate entities.
12. For each entity, list what you need to know about it. Those are the columns.

13. Give every table an id primary key.
14. Work out how entities relate: one-to-many, many-to-many, one-to-one.
15. Add foreign keys for one-to-many. Create a join table for many-to-many.
16. Check third normal form: every fact stored exactly once, no repeating groups.
17. Choose sensible types and constraints - NOT NULL, UNIQUE, sensible lengths.
18. Draw the ER diagram and put it in your README.

```
-- schema.sql
CREATE TABLE users (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  name          VARCHAR(100)  NOT NULL,
  email         VARCHAR(150)  NOT NULL UNIQUE,
  password_hash VARCHAR(255)  NOT NULL,
  role          ENUM('customer','owner') NOT NULL DEFAULT 'customer',
  created_at    TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE services (
  id               INT AUTO_INCREMENT PRIMARY KEY,
  name             VARCHAR(120)   NOT NULL,
  duration_minutes INT            NOT NULL,
  price            DECIMAL(10,2)  NOT NULL,
  is_active        BOOLEAN        NOT NULL DEFAULT TRUE
);

CREATE TABLE bookings (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  user_id     INT       NOT NULL,
  service_id  INT       NOT NULL,
  starts_at   DATETIME  NOT NULL,
  status      ENUM('confirmed','cancelled','completed') NOT NULL DEFAULT 'confirmed',
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

  FOREIGN KEY (user_id)    REFERENCES users(id)    ON DELETE CASCADE,
  FOREIGN KEY (service_id) REFERENCES services(id) ON DELETE RESTRICT,
  UNIQUE KEY one_booking_per_slot (service_id, starts_at)
);
```

Note the UNIQUE key: double bookings are prevented by the database, not by hoping the code is correct.

BEST PRACTICE

Enforce rules in the database wherever you can. Application code has bugs and gets bypassed; a UNIQUE constraint does not. That last line is the entire double-booking problem solved.

Chapter 7 - Stage 5 - Development

Build Order That Works

19. Set up the repository, .gitignore and .env.example before writing any code.
20. Create the database from schema.sql and add seed data you can develop against.
21. Build the database connection layer once, reading credentials from the environment.
22. Build registration and login. Everything else depends on knowing who the user is.

23. Build the main entity's create and read. Get one full journey working end to end.
24. Add update and delete, with an authorization check on each.
25. Build the user dashboard.
26. Build the admin views.
27. Build the public-facing pages.
28. Style everything properly - do not leave this to the last day.
29. Add validation and error handling across every form.
30. Refactor the duplication you have accumulated.

WHY THIS MATTERS

Getting one journey working end to end early - register, log in, create, see it listed - is the single best decision you can make. It proves the whole stack connects, and it means that if you run out of time you still have something that works.

Non-Negotiable Patterns

```
// Every query, without exception, is prepared.
$stmt = $pdo->prepare('SELECT * FROM bookings WHERE user_id = ?');
$stmt->execute([$userId]);
$bookings = $stmt->fetchAll();

// Passwords are hashed, never stored or logged in the clear.
$hash = password_hash($password, PASSWORD_DEFAULT);
if (password_verify($input, $user['password_hash'])) { /* ok */ }

// Output is escaped, every time, without exception.
echo htmlspecialchars($booking['notes'], ENT_QUOTES, 'UTF-8');

// Authorization is checked on the SERVER, for every action.
if ($booking['user_id'] !== $_SESSION['user_id'] && $_SESSION['role'] !== 'owner') {
    http_response_code(403);
    exit('Not allowed');
}
```

These four patterns are checked in marking. Missing any one of them costs marks in the security band.

Working in Small Commits

Commit each finished step. Your Git history is assessed, and more importantly it is your safety net: when something breaks on day eight, a clean history lets you find exactly when it broke. A single commit called 'final project' tells your marker that you did not use version control, you used a backup.

Chapter 8 - Stage 6 - Testing

Testing is not clicking through the parts you know work. It is deliberately trying to break your own system, which requires a different mindset from building it.

Category	What you try	Example

Category	What you try	Example
Happy path	The intended journey, as a new user	Register, log in, book, see it listed
Invalid input	Empty, too long, wrong type, wrong format	A booking date in the past
Hostile input	Injection and script payloads	' OR '1'='1 and <script>alert(1)</script>
Authorization	Reaching another user's data	Change the id in the URL to someone else's booking
Edge cases	Boundaries and unusual timing	Two people booking the same slot at once
Responsive	Every breakpoint on a real device	360px on an actual phone, not just DevTools
Error handling	Break things deliberately	Stop MySQL and load a page

```
# TESTING.md

| # | Test case                          | Expected                  | Actual | Pass |
|---|------------------------------------|---------------------------|--------|------|
| 1 | Register with a valid email        | Account created, logged in| As expected |
Yes |
| 2 | Register with an existing email    | Clear error, no duplicate | As expected |
Yes |
| 3 | Book a slot already taken          | Rejected with a message   | Was allowed | No
-> fixed in a3f9c21 |
| 4 | Cancel another user's booking by URL| 403 Forbidden            | As expected |
Yes |
| 5 | Submit <script>alert(1)</script>   | Escaped and shown as text | As expected |
Yes |
```

Record failures and how you fixed them. A TESTING.md showing bugs you found and fixed scores better than one showing everything passed first time.

NOTE

Test case 3 above is the honest kind. Finding a real bug, recording it, fixing it and referencing the commit demonstrates exactly the professional behaviour this project is assessing.

Chapter 9 - Stage 7 - Deployment

31. Confirm no credentials exist anywhere in the repository or its history.
32. Provision hosting that supports your PHP version, and create the production database.
33. Import schema.sql and load minimal, realistic seed data for the demonstration.
34. Create the production environment configuration on the server.
35. Point the web root at your public folder so source is never served as text.
36. Turn display_errors off and log_errors on.
37. Enable HTTPS and force the redirect from http.
38. Walk the entire happy path on the live site, on a phone that is not yours.
39. Set up a database backup before you show the site to anybody.
40. Put the live URL at the top of your README.

COMMON MISTAKE

Deploy by day ten, not on the final evening. Deployment always surfaces problems - a different PHP version, case-sensitive filenames, missing extensions, file permissions. You need time to fix them, and that time has to exist before the deadline, not after it.

Chapter 10 - Documentation and the README

Your README is the front door. Assume the reader is an employer with sixty seconds, or a developer inheriting your project on Monday. Both need different things and both are served by the same structure.

````
# Salon Booking System

Live: https://salonbooking.example.com

## The problem
Two sentences on the real problem and who has it.

## What it does
- Customers browse services and book available slots
- Owners see the day's schedule on one screen
- Owners manage services and staff

## Screenshots
![Customer booking](docs/booking.png)
![Owner dashboard](docs/dashboard.png)

## Built with
HTML, Tailwind CSS, JavaScript, PHP 8.2, MySQL 8

## Database
![ER diagram](docs/er-diagram.png)

## Running it locally
```bash
git clone https://github.com/you/salon-booking.git
cd salon-booking
cp .env.example .env      # then fill in your values
mysql -u root -p < schema.sql
php -S localhost:8000 -t public
```

## Environment variables
| Variable    | Purpose              |
|-------------|----------------------|
| DB_HOST     | Database hostname    |
| DB_NAME     | Database name        |

## Testing
See TESTING.md.

## Known limitations and future work
Honest, specific, and it is a strength not a weakness.

## Author
````

```
Your Name - Get Techy With Lucky, Web Development Masterclass
```

Every section here earns its place. The setup commands must actually work on a clean machine - test them.

BEST PRACTICE

The 'known limitations' section makes you look more professional, not less. Every real system has them. Naming yours shows you understand your own work and know what you would do next.

Chapter 11 - Presenting and Defending Your Work

Minute	Cover	Avoid
0-1	The problem and who has it	Starting with your tech stack
1-2	What you built, in one sentence, then the live demo	Reading your slides aloud
2-6	Walk the main user journey live	Showing code the audience cannot read
6-8	One interesting technical decision and why	Apologising for what is unfinished
8-10	Limitations, what you learned, questions	Running over time

Questions You Will Be Asked

- Why did you choose this problem, and who did you talk to about it?
- Walk me through what happens when a user submits this form.
- How do you stop somebody cancelling another person's booking?
- Why is this table structured this way rather than as one big table?
- What was the hardest bug, and how did you find it?
- What would you do differently with another two weeks?

NOTE

'I do not know, but here is how I would find out' is a good answer. Inventing something is not. Examiners and interviewers are testing your reasoning, not your ability to appear infallible.

Chapter 12 - Important Terminology

These terms span the whole masterclass. If any are unfamiliar, revisit that week's guide before you start building - the capstone assumes all of them.

Term	Definition	In Plain Words	Example / Use Case
Capstone project	A final project drawing together everything learned in a course.	The one where you prove it all works.	A full-stack booking system
Problem statement	A clear description of the problem the software solves and for whom.	Why this should exist.	'Salons lose bookings taken over WhatsApp'
Scope	The agreed boundary of what will	What is in and what is out.	Bookings in, payments out

Term	Definition	In Plain Words	Example / Use Case
	and will not be built.		
Scope creep	Uncontrolled growth of scope during a project.	'Could it also just...'	Adding a chat feature in week two
MVP	Minimum viable product - the smallest version that genuinely solves the problem.	The least you can build that still works.	Book, view, cancel - nothing else
Target user	The specific person the product is designed for.	Who this is actually for.	A salon owner with one receptionist
User story	A requirement expressed from the user's perspective.	A feature described by who and why.	'As an owner I want to see today's bookings'
Acceptance criteria	The conditions that must be true for a story to be considered done.	How you know it is finished.	'Cancelling frees the slot immediately'
Functional requirement	Something the system must do.	A capability.	'The system emails a confirmation'
Non-functional requirement	A quality the system must have.	How well it must behave.	'Loads in under 3s on 3G'
Wireframe	A low-detail sketch of a screen's layout.	A rough drawing of the page.	Boxes and labels, no colour
Mockup	A higher-fidelity visual design of a screen.	What it will actually look like.	A Figma design
User flow	The path a user takes through the system to achieve a goal.	The steps from start to done.	Browse -> select slot -> confirm
System architecture	How the parts of the system fit together.	The map of your application.	Browser -> PHP -> MySQL
Three-tier architecture	Separating presentation, application logic and data.	Front end, back end, database.	HTML/CSS/JS, PHP, MySQL
Entity	A thing the system stores information about.	A noun that becomes a table.	User, Booking, Service
Attribute	A property of an entity.	A column.	Booking.start_time
Relationship	How two entities are connected.	How tables link.	A user has many bookings
ER diagram	A picture of entities, attributes and relationships.	The database, drawn.	Boxes joined by lines
Primary key	A column uniquely identifying each row.	The row's ID.	bookings.id
Foreign key	A column referencing the primary key of another table.	The link between tables.	bookings.user_id
Normalization	Organising tables to remove duplication and inconsistency.	Store each fact once.	Third normal form
Schema	The complete structure of the database.	The blueprint of your tables.	Exported as schema.sql
Migration	A versioned, repeatable change to the database structure.	A recorded schema change.	add_status_to_bookings.sql
Seed data	Example data loaded so the system can be demonstrated.	Realistic sample content.	Ten fake bookings
CRUD	Create, read, update, delete - the four basic data operations.	The four things you do to data.	Add, view, edit and cancel a booking

Term	Definition	In Plain Words	Example / Use Case
Authentication	Verifying who a user is.	Proving you are you.	Email and password login
Authorization	Deciding what an authenticated user may do.	What you are allowed to touch.	Only an admin may delete
Session	Server-side state identifying a logged-in user across requests.	How the server remembers you.	PHP $_SESSION
Password hashing	Storing an irreversible transformation of a password.	Storing proof, not the password.	password_hash()
Prepared statement	A parameterised query that separates SQL from data.	The defence against SQL injection.	PDO with bound parameters
SQL injection	An attack inserting SQL through unsanitised input.	Tricking the database with input.	' OR '1'='1
XSS	Cross-site scripting - injecting script through unescaped output.	Someone else's JavaScript on your page.	Escape output with htmlspecialchars()
CSRF	Forcing a logged-in user's browser to submit an unwanted request.	A forged request using your session.	Defended with a per-form token
Validation	Checking that input is acceptable before using it.	Is this data allowed?	Rejecting a booking in the past
Sanitisation	Cleaning input or escaping output so it cannot cause harm.	Making data safe to use.	Escaping before display
Unit test	An automated test of one small piece of logic.	Testing one function.	Does the total calculate correctly?
Manual test	A person following steps and checking the result.	Clicking through it yourself.	A written test plan
Test case	One specific scenario with inputs and an expected result.	One thing you check.	'Booking a taken slot is rejected'
Edge case	An unusual input at the boundary of what is expected.	The weird one that breaks things.	Booking at exactly midnight
Regression	A previously working feature that a change has broken.	You fixed one thing and broke another.	Login stops working after a refactor
Bug report	A record of a defect with steps to reproduce it.	How to make it go wrong again.	Steps, expected, actual
Deployment	Making the system available in a live environment.	Putting it online.	Uploading to hosting
Environment variable	Configuration supplied from outside the code.	A per-environment setting.	DB_PASSWORD
Technical debt	Shortcuts taken now that cost more to fix later.	Borrowed time you repay with interest.	Duplicated code you meant to tidy
Refactoring	Improving code structure without changing behaviour.	Tidying without breaking.	Extracting a repeated block into a function
Code review	Another developer reading your changes before merge.	A second pair of eyes.	Comments on a pull request
Documentation	Written explanation of what the system is and how to run it.	The manual.	README.md
Demo	A prepared walkthrough of the working system.	Showing it, not describing it.	A five-minute presentation

Term	Definition	In Plain Words	Example / Use Case
Portfolio	The collection of work you show employers.	Your evidence.	Pinned GitHub repositories

Chapter 13 - Common Mistakes and How to Avoid Them

Mistake	What goes wrong	The fix
Scope too large	Nothing works end to end at the deadline	Define an MVP on day one and defend it
Building before designing the database	Painful rework once ten pages depend on the schema	Finish the ER diagram before writing queries
Leaving styling to the last day	An unusable interface and no time to fix it	Style each feature as you finish it
Deploying on the final evening	Deployment problems with no time to solve them	Deploy by day ten
One giant commit	No history, no safety net, marks lost	Commit each completed step
Only testing the happy path	Bugs and vulnerabilities found by the marker	Test invalid, hostile and authorization cases
Authorization only hidden in the UI	Anyone can act by changing a URL	Check permissions server-side on every action
Credentials in the repository	Security failure, marks lost, real risk	Environment variables from the first commit
README written last	Vague, incomplete, setup steps that do not work	Write it as you go and test the steps
Adding features in the final days	New bugs with no time to fix them	Freeze features; spend the last days testing and polishing

Chapter 14 - Security Checklist Before You Submit

Go through this line by line on the live site. Every item is checked during marking.

Check	How to verify
Every query uses prepared statements	Search the codebase for string concatenation inside SQL
Passwords hashed with password_hash()	Look in the database - you must not be able to read any password
All output escaped	Submit <script>alert(1)</script> in every text field and confirm it displays as text
Authorization checked server-side	Log in as user A and try to open user B's record by URL

Check	How to verify
No credentials in the repository	Search the full history, not just the current files
display_errors off in production	Trigger an error and confirm no file path is shown
HTTPS enforced	Visit the http:// URL and confirm it redirects
Session destroyed on logout	Log out, then press back and try to reach a protected page
Validation on the server, not only the browser	Disable JavaScript and submit an invalid form
Source files not served as text	Try to open a .php file directly and confirm it does not show source

Chapter 15 - Marking Rubric

Band	What earns it
Distinction (80-100)	Everything works end to end, secure throughout, deployed, excellent documentation, confident defence, thoughtful extras
Merit (65-79)	Core system complete and secure, deployed, good documentation, sound presentation, minor gaps
Pass (50-64)	Main journey works, basic security present, deployed or clearly runnable, adequate documentation
Fail (below 50)	Core journey incomplete, security patterns missing, not deployed, or documentation too thin to run the project

The weighting across criteria is in the project brief that follows.

Chapter 16 - Knowledge Check - Readiness Quiz

Take this before you start building. It is a readiness check rather than a recall test - anything you get wrong is something to revisit before it costs you days.

Pass mark: 70% | *Answers are at the end of this chapter.*

Q1. What should a problem statement contain?

a) The technologies you plan to use
b) The problem, who has it, and what it currently costs them
c) A list of every feature you will build
d) Your database schema

Q2. You have three weeks and a long feature list. What is the correct approach?

a) Start all features and finish whichever you can
b) Define an MVP that fully solves the core problem, build that completely, then add extras if time allows

c) Build the most impressive feature first
d) Reduce quality across all features so they all fit

Q3. Where must a user's password be hashed?

a) In JavaScript before sending it
b) On the server, before storing it in the database
c) In the database using a view
d) It should be encrypted, not hashed, so it can be recovered

Q4. What is the single most important defence against SQL injection?

a) Hiding error messages
b) Prepared statements with bound parameters
c) Removing apostrophes from input
d) Using POST instead of GET

Q5. A project with more features always scores better than one with fewer.

True / False

Q6. Documentation can be written at the end, after the code is finished.

True / False

Q7. Explain what an MVP is and describe how you would cut your own capstone down to one.

Short answer - write two or three sentences.

Q8. Your booking system lets a logged-in user cancel a booking. Describe the authorization check you must perform and what happens without it.

Short answer - write two or three sentences.

Q9. Describe three tests you would run before declaring your capstone finished.

Short answer - write two or three sentences.

Q10. Why does the README matter as much as the code for your portfolio?

Short answer - write two or three sentences.

Answer Key

Mark your own work honestly - the goal is to find your gaps, not to score well.

#	Answer	Why
Q1	b - The problem, who has it, and what it costs them	A problem statement is about the problem, not the solution. Technology choices come later and should follow from the requirements, not lead them.
Q2	b - Define and complete an MVP first	A small system that works end to end is worth far more than a large one that half works. Examiners and employers both judge completeness over ambition.
Q3	b - On the server, before storing	Hashing in the browser means the hash becomes the password in transit. And passwords are hashed, never encrypted - you must never be able to recover the original.

#	Answer	Why
Q4	b - Prepared statements with bound parameters	Prepared statements send the query structure and the data separately, so input can never be interpreted as SQL. Filtering characters is a workaround people always get wrong.
Q5	False	A complete, working, well-documented small system beats an ambitious broken one every time. Half-finished features are evidence of poor scoping, not of ambition.
Q6	False	Written last, it is written from memory and always misses the details that mattered. Write the README as you go, and update it as decisions change.
Q7	An MVP is the smallest version that genuinely solves the core problem for the target user. To cut down, list every feature, identify the single user journey that delivers the main value, keep only what that journey requires, and move everything else to a 'future work' section in the README.	Scoping is the skill this stage teaches. Most failed student projects failed at scope, not at coding.
Q8	Before cancelling, the server must confirm the booking belongs to the logged-in user, or that the user is an admin. Without that check, anyone could cancel anyone else's booking by changing the ID in the URL - an insecure direct object reference.	Authentication without authorization is one of the most common real vulnerabilities. Knowing who someone is does not tell you what they may touch.
Q9	Test the full happy path end to end as a new user. Test invalid and hostile input on every form, including empty fields, wrong types and a script tag. Test authorization by attempting to access another user's data by manipulating a URL.	Students test the happy path and stop. The other two categories are where the marks and the vulnerabilities are.
Q10	It is the first and often only thing a reviewer reads. It demonstrates that you can communicate technical	Employers spend under a minute on a repository. The README is what they spend it on.

#	Answer	Why
	decisions, and it determines whether anyone can actually run your project. A good project nobody can run reads as a broken project.	

Chapter 17 - Project Brief - The Capstone Specification

Objective

Design, build, test, document, deploy and present a complete full-stack web application that solves a genuine problem for a real category of user, using everything covered in Weeks 1 to 7.

Scenario

You are the sole developer for a small organisation in your community. They have a real operational problem currently handled with paper, WhatsApp or a spreadsheet. You will take the project from a blank folder to a live, documented system, and then defend your design decisions in a presentation.

Requirements

- A public-facing area that anybody can view without logging in.
- User registration and login with securely hashed passwords.
- At least two roles with genuinely different permissions, such as user and administrator.
- Complete create, read, update and delete functionality for the system's main entity.
- A database of at least four related tables using proper foreign keys.
- Server-side validation on every form, in addition to any client-side validation.
- A dashboard showing the logged-in user information relevant to them.
- An administrator view for managing the system's data.
- A fully responsive interface working from 320px upward.
- Meaningful error handling: nothing crashes, and users get useful messages.

Expected Output

A deployed, publicly reachable application served over HTTPS; a GitHub repository with a readable commit history; complete documentation including an ER diagram and setup instructions; and a five to ten minute presentation demonstrating the system and explaining your decisions.

Technical Requirements

- Front end built with HTML, CSS and JavaScript. Tailwind is permitted and encouraged.
- Back end in PHP, with MySQL for storage.
- All database access through prepared statements without exception.
- Passwords hashed with password_hash(); no plaintext password stored or logged anywhere.
- All output escaped before display to prevent cross-site scripting.
- Every configurable value in environment variables; no credentials anywhere in the repository.
- Sessions used for authentication, with logout that genuinely destroys the session.
- Authorization checked on the server for every action, never only hidden in the interface.
- Git history showing incremental work across the whole project, not one large final commit.

Submission Instructions

41. Push the complete project to a public GitHub repository named after your project.
42. The README must include: the problem statement, features, technologies, ER diagram, setup instructions, live URL and screenshots.
43. Include schema.sql so the database can be recreated from scratch.
44. Include a TESTING.md recording your test cases and their results.
45. Deploy the application and confirm the live URL works from a device that is not yours.
46. Submit the repository link and live URL on the Week 8 final project page in the student portal.
47. Prepare a five to ten minute presentation and be ready to answer questions about your decisions.

Evaluation Criteria

Criterion	What Earns Full Marks	Weight
Functionality	Every stated feature works; the core journey is complete end to end	25%
Database design	Normalised, correct keys and relationships, ER diagram matches reality	15%
Security	Prepared statements, hashed passwords, escaped output, server-side authorization	15%
Code quality	Organised, readable, no large duplicated blocks, sensible naming	10%
User interface	Responsive, accessible, consistent, genuinely usable on a phone	10%
Deployment	Live over HTTPS and working reliably	10%
Documentation	A stranger can understand, install and run it unaided	10%
Presentation	Clear demonstration and confident justification of decisions	5%

Bonus Challenges

- Email notifications for a meaningful event, such as a booking confirmation.
- Search and filtering across the main data set.
- Exporting a report to CSV or PDF.
- A simple analytics dashboard with a chart drawn from real data.
- Containerising the project so it runs with a single docker compose up.
- A CI workflow that checks the build on every push.

Chapter 18 - Summary and What Comes Next

- Choose a problem small enough to finish, then make what you finish excellent.
- Write the proposal before any code. The out-of-scope list is what protects you later.
- Design the database before writing queries, and enforce rules in the schema where you can.
- Get one journey working end to end early. It proves the stack connects and guarantees you have something.
- Prepared statements, hashed passwords, escaped output and server-side authorization are non-negotiable.
- Test hostile input and authorization bypass, not just the happy path.
- Deploy by day ten. Deployment always surprises you, and you need time to react.
- The README is the front door. Write it as you go and confirm the setup steps actually work.
- In the presentation, lead with the problem and demonstrate the working system.

After the Masterclass

Finishing this project makes you a junior developer with a portfolio, which is more than most people applying for their first role have. What you do next matters.

- **Keep the project alive.** Fix a bug, add one item from your future work list, keep the commit graph moving.
- **Build a second project, differently.** Pick a stack you have not used. The second one is where the fundamentals become portable.
- **Contribute to something open source.** Start with documentation fixes. Getting one pull request merged into somebody else's project teaches more than a month of tutorials.
- **Write about what you built.** A short post explaining one problem you solved is worth more to an employer than a certificate.
- **Keep learning in public.** The habits from this course - small commits, real READMEs, deployed work - are what compound.

You started eight weeks ago not knowing what an HTML tag was. You are finishing with a deployed, secured, documented full-stack application that solves a real problem for real people. That is not a small thing. Learn, build, connect and keep growing.

Further Reading and References

Documentation changes faster than any printed guide. Learning to read the official docs is part of the skill you are building.

- **The Twelve-Factor App -** https://12factor.net/
- **OWASP Top Ten -** https://owasp.org/www-project-top-ten/
- **PHP: password_hash documentation -** https://www.php.net/manual/en/function.password-hash.php
- **PHP: PDO prepared statements -** https://www.php.net/manual/en/pdo.prepared-statements.php
- **MySQL: normalization and database design -** https://dev.mysql.com/doc/
- **Make a README - structure and examples -** https://www.makeareadme.com/
- **Choose a License -** https://choosealicense.com/
- **Conventional Commits -** https://www.conventionalcommits.org/